

Cory Carter

June 28, 2026

CS 470 Final Reflection

YouTube Presentation Link:

<https://www.youtube.com/watch?v=lu2rrRJ3Rgs>

Experiences and Strengths

CS 470 helped me gain practical experience with cloud computing and modern web application development. Before taking this course, I understood the basics of full stack development, but I had very little experience deploying applications in the cloud. Throughout the course, I learned how to migrate a traditional MEAN stack application into a cloud-native architecture using AWS services.

Some of the most valuable skills I developed include working with Amazon S3 for static website hosting, API Gateway for managing HTTP requests, AWS Lambda for serverless computing, DynamoDB for NoSQL data storage, IAM roles and policies for security, and Docker for containerization. I also gained experience testing APIs, troubleshooting cloud deployments, and understanding how different AWS services communicate with one another. These are all skills that are commonly used in today's software industry and make me a stronger candidate for future software engineering positions.

One of my strengths as a software developer is my willingness to learn new technologies and solve problems through research and testing. Throughout this course I encountered several deployment and configuration issues, but I continued troubleshooting until I successfully completed each milestone. This process improved both my technical skills and my confidence when working with unfamiliar cloud technologies.

After completing this course, I feel prepared to pursue entry-level software engineering, cloud engineering, or full stack developer positions. I also feel more comfortable working with cloud platforms and understand how cloud-native applications are designed, deployed, and maintained. Since I currently work for Lockheed Martin and am completing my software engineering degree, these new cloud computing skills will help me work toward my long-term goal of becoming a software engineer within my company.

Planning for Growth

One of the biggest advantages of using a cloud-native architecture is the ability to scale individual services independently instead of scaling an entire application. If this application continues to grow, I would continue using a microservices and serverless approach because it

allows each service to perform one specific responsibility while remaining easier to maintain and update.

To handle increased traffic, AWS Lambda can automatically scale by creating additional function instances whenever more requests arrive. API Gateway can distribute incoming requests efficiently, while DynamoDB can scale automatically to handle increased read and write operations. This reduces the need for manually provisioning servers and allows the application to respond to changing demand with minimal administrative effort.

Error handling would also become more important as the application grows. I would implement more detailed logging with Amazon CloudWatch to monitor Lambda executions, identify failed requests, and quickly diagnose issues. Retry mechanisms, input validation, proper exception handling, and meaningful API responses would improve reliability while making the application easier to maintain.

Cost prediction is another important factor when planning future growth. AWS provides cost estimation tools that allow developers to estimate monthly expenses based on expected usage. Services like AWS Cost Explorer and the AWS Pricing Calculator help estimate the cost of Lambda executions, API requests, storage, and database usage before deployment.

Between containers and serverless computing, containers generally provide more predictable costs because they typically run continuously regardless of application usage. Serverless computing is less predictable because costs increase and decrease based on the number of requests and execution time. However, for applications with inconsistent or unpredictable traffic, serverless computing is often more cost-effective because you only pay when code is actually running.

Both approaches have advantages and disadvantages. Serverless applications require less infrastructure management, automatically scale with demand, and reduce operational overhead. However, they may experience cold starts and can become more expensive if workloads remain consistently high. Containers provide greater control over the runtime environment and perform well for long-running applications, but they require more administration, monitoring, and capacity planning.

Elasticity is one of the most important cloud concepts because it allows computing resources to automatically increase or decrease as demand changes. Combined with the pay-for-service pricing model, organizations can avoid paying for unused infrastructure while still supporting periods of heavy traffic. These cloud features make it easier to plan for future growth because applications can expand without requiring significant hardware investments or manual server management.

Overall, CS 470 gave me valuable hands-on experience with cloud technologies and demonstrated how modern applications are designed for scalability, reliability, and cost efficiency. The knowledge I gained throughout this course has strengthened my technical skills and prepared me for future software engineering opportunities involving cloud-native application development.

